

InstanceRecord 拆分可行性与回归控制研究

1. 结论与建议

`_instance_contracts.py` 可以拆分，但最有价值的第一步是分离校验职责，而不是立即把类或字段搬到另一个文件。建议保留 `InstanceRecord` 的类定义、38 个字段、CSV 接口及公开身份，先在同一文件中提取只读的家族校验函数。这样可将 `__post_init__()` 从 381 行降到 163 行，降低理解一次构造所需同时追踪的分支数量；文件总行数会略增，这是职责显式化的成本，不应作为失败判据。^{[1][9]}

进一步跨文件分离也具备可行性：隔离原型可将原文件缩为约 437 行，新增约 540 行的家族规则模块。然而，原型在无参数调用私有校验函数的 `typing.get_type_hints()` 时出现类型名解析差异。项目当前没有发现该私有反射用法，公开类及 CSV 方法的类型解析不受影响；仍应把它作为明确的兼容取舍，而非忽略后宣称完全等价。^{[9][12]}

研究以本地提交 `2c4c3de` 的源码为基线。证据包括源码依赖、现有行为测试、独立进程中加载的临时提取原型，以及固定的 8,790 项输入情形。三个候选在所比较的公开行为上均没有差异；刻意打乱校验顺序的负对照产生了 13 项差异，说明比较能够识别错误优先级变化。这个结果支持分阶段实施，不能证明任何未来改动都不会产生回归。^[9]

推荐顺序是：固定公开契约与多错误案例；同文件提取家族校验；完成真实代码的等价性、类型检查和独立审阅；再决定是否跨文件分离。不要同时修改接受范围、统一异常类型、去除重复证书检查，或建立通用校验框架。历史研究配置、数据和 DUAL 的源码约束继续保持。

2. 当前文件实际承担什么职责

文件共 952 个物理行，包含空行。顶部定义 schema 版本和三项 P4.3 家族元数据；54-338 行包含三个参数解析辅助函数、P4.3 参数检查及配对 `uniform` 参数检查；342 行开始定义 `frozen`、`slots` 的 `InstanceRecord`。类内还有构造校验、CSV 输出和 CSV 输入。最后一行把公开模块身份固定为 `maxcover.contracts`。^[1]

区域	当前范围	主要责任	建议处理
schema 与家族元数据	22-43 行	版本、研究问题、来源、配对家族集合	schema 保留；家族元数据后续与规则同归属
参数校验辅助函数	54-338 行	参数类型、键集合、容量与跨参数关系	第一阶段保留；跨文件时与家族规则一起移动
类型形状	341-423 行	38 个字段、顺序、默认值、CSV 字段	保留定义和公开身份
通用构造校验	425-586 行	类型、维度、JSON、比率、通用证书一致性	首批留在原位置 and 原顺序
家族校验	587-805 行	legacy/v2、P4.3、paired uniform、其他随机族	推荐提取的完整责任块
CSV 与公开模块身份	807-952 行	输出格式、输入解析、类型定位	首批不移动、不改写

当前家族规则的维护负担来自两处：参数关系在前部辅助函数中，来源、研究问题、耦合字段和证书关系在 `__post_init__()` 后半段。修改一个家族时，必须同时检查这两组规则。仅把顶部辅助函数放入新文件，虽然缩短原文，却会使维护者在两个文件之间跳转，且不能缩短 381 行的构造校验。因此它不是推荐的独立首批交付。^[1]

家族分支并不对称。`dominated_heavy` 的参数辅助函数会提前返回，随后仍有专门的证书检查；`paired_uniform` 的识别条件同时涉及参数键和研究问题标识；`fixed_size` 又区分有无配对种子。把这些分支压成统一注册表或通用参数模式，容易改变适用范围和执行顺序。^[1]

3. 内部依赖与结果影响链

生产侧由 `benchmark_planning_instance_record()` 结合生成器信息、结构测量与证书创建记录。这个过程包含自己的来源和耦合检查，随后才调用 `InstanceRecord`。读取侧则通过 `InstanceRecord.from_csv_row()` 先解析 CSV 字符串，再进入同一个构造流程。两条入口承担的责任不同，不能只测试其中之一。^{[3][4]}

构造过程的顺序可以概括为：schema 和基本身份 → 整数与维度 → JSON 解析及规范化 → 比率与计数一致性 → 通用证书与选择索引规范化 → 家族规则。CSV 入口在这些步骤之前，还会检查字段集合、解析 schema 和 `optimum_selected`。这些先后关系决定同一行同时出现多种错误时最先报告什么。^[1]

记录随后进入统计、关联和报告，也进入恢复时的已有实例比对。`benchmark_artifacts` 会把记录转换为规范 CSV，再读回类型化记录；恢复读取还会与按配置重建的预期实例记录比较。因此，一处看似只是格式调整的变化，也可能改变 `instances.csv` 的字节、结果匹配或恢复的拒绝行为。^{[3][4]}

输出验证脚本也使用这一记录类加载实例。它能检查跨文件一致性，但若生产和验证同时使用改错后的构造逻辑，两边可能一致地接受错误数据。因此，验证脚本通过只能作为辅助证据；迁移验收还需要独立进程中的旧新版差分、已知答案以及人工规定的拒绝案例。^{[4][9]}

`contracts.py` 和包根目录重新导出记录类与家族常量。常量映射当前是只读的 `MappingProxyType`，配对家族为 `frozenset`。跨文件时应移动定义并保持原位置的同一对象导出，避免复制可漂移的表；同文件提取则不需要改变这条依赖。^{[1][2]}

4. 必须保护的兼容边界

第一类是类型形状：类对象身份、`maxcover.contracts.InstanceRecord` 路径、字段和 CSV 列顺序、默认值、`__slots__`、`__match_args__`、`frozen` 行为及序列化状态。现有兼容测试明确检查其中多项；保留类定义可以大幅缩小这类风险。将类型拆成多个继承层、为不同家族增加子类，或换成另一个校验库都超出本批目标。^{[2][5][10]}

第二类是两处有意的规范化写入。JSON 在解析后立即被转换为排序且紧凑的字符串；`optimum_selected` 则在合法性检查后转换为 `tuple`。这些 `object.__setattr__` 对新实例的写入是 `frozen dataclass` 构造过程的一部分。家族校验必须继续接收已解析的参数对象及规范化后的字段，不能重新解析或重新设置它们。^[1]

第三类是错误先后与错误类别。基本整数检查会产生 `TypeError`；其他规则可能产生 `ValueError`。比率字段碰到不支持的对象时，也可能由标准数学函数产生 `TypeError`。纯拆分不应顺手把所有错误包装成统一 `ValueError`；否则现有调用方的拒绝行为会改变。^{[1][9]}

第四类是证书资格。通用检查要求四个证书字段成组出现；家族分支随后规定具体证书是否允许、来源和见证是什么。不得先把一个“看起来完整”的证书当作适用于任意家族。前两批比较夹具曾因证书构造约束而需要修正，这也说明仅捕获任意异常不足以证明命中了目标校验。^{[1][6]}

第五类是 pickle 与 dataclasses.replace 的差别。本地运行时实测及 Python 官方说明一致：replace 会经构造流程再次调用 `__post_init__`；frozen/slots dataclass 的默认反序列化状态恢复则不执行这套构造校验。因此“旧 pickle 能读取”证明的是兼容性，不能替代记录有效性验证，也不能在这次重构中新增反序列化校验并把它称作纯迁移。^{[9][10][11]}

第六类是冻结研究。DUAL 的 `CODE_PATHS` 覆盖整个 `src/maxcover`，新增或改写记录校验文件也会改变源码匹配结果。应继续使用已隔离恢复的旧源码复现历史研究，新维护版本使用小样本验证；不修改旧 `code_revision` 或缩小源码检查路径。^[8]

5. 拆分方案比较

本研究比较了三种实际加载过的原型。行数反映临时原型的物理行，正式实现因注释和排版可能略有变化，不作为代码规模承诺。

方案	原文件 / 新模块	<code>__post_init__</code>	维护收益	主要取舍
A: 同文件提取家族校验	958 / 无	163 行	通用构造与家族规则分开；无新增运行时依赖	文件不变短，但主要长方法变短
B: 只移出参数辅助函数与元数据	654 / 318 行	381 行	原文件变短	家族规则更分散；引入类型注解解析问题
C: 家族参数与资格规则一起移出	437 / 540 行	163 行	一个模块集中维护家族规则	需要明确跨模块类型依赖与私有反射约定

建议先做 A，再评估 C；不建议单独实施 B。A 把最危险的变化限制为一个只读责任块的提取：类和所有运行时名字仍在原模块，JSON 和选择索引的规范化保持原位。它为后续家族规则整理提供了可测试边界，也无需复制字段声明。^[9]

C 中的新模块可命名为 `_instance_family_validation.py`，包含家族元数据、参数辅助函数及 `_validate_instance_family(record, parameters, certificate_values)`。InstanceRecord、schema 检查和 CSV 方法仍在 `_instance_contracts.py`。不要让新模块运行时反向导入公共 `contracts`，否则容易形成部分初始化的循环依赖。^{[1][2]}

研究原型采用 `TYPE_CHECKING` 解决静态类型引用，公开类和 CSV 方法的类型解析保持一致，但两个迁移后的私有校验函数直接调用 `get_type_hints()` 会缺少 InstanceRecord 的运行时名字。显式提供类型命名空间可以恢复解析。仓库搜索未发现该私有调用，但这个差异仍需在未来正式方案中说明；若要求所有私有反射也保持原样，应停留在 A，先决定类型边界，不能用动态全局注入或大规模字段 Protocol 勉强绕过。^{[9][12]}

6. 隔离原型和输入探针的证据

输入来自既有 75 行实例兼容夹具的代表记录，以及少量按现有配置生成的 QA 记录；没有求解正式研究实例。共选取 13 个代表，包括普通 uniform、high_overlap、dominated_heavy、mixed_cluster、long_tail、adversarial v2、配对 fixed_size/uniform、duplicate_heavy、legacy adversarial、非配对/唯一 fixed_size 和自定义来源记录。^{[5][6][9]}

每个版本执行相同的 8,790 项调用情形：4,940 项直接构造字段变异、3,458 项 CSV 字段变异、273 项嵌套参数变异、78 项多错误组合，另有合法输入与规范化案例。这是确定性的工程探针，包含相同值和重复情形，不是 8,790 个独立随机样本，也不是缺陷率或失败概率估计。^[9]

基线接受 979 项、拒绝 7,811 项。A、B、C 与基线逐项一致，比较包含接受/拒绝、异常类型与信息、成功后的规范 CSV 和 hash；调用者提供的输入对象未变异。类签名、字段顺序、slots、模式匹配字段、公开身份和旧 protocol-4/5 pickle 状态也一致。C 的家族函数展开并将参数名恢复后，整个 __post_init__ 的 AST 与基线一致。^[9]

检查	A	B	C
8,790 项输入行为差异	0	0	0
公开类型形状与旧 pickle 状态	一致	一致	一致
公开类型注解解析	一致	一致	一致
私有 helper 直接类型注解解析	一致	NameError	NameError
选取的 15 项现有行为测试	通过	未运行	通过

基线也运行了同一组 15 项测试。它们覆盖公开导出、CSV、frozen/slots、pickle/spawn、P4.3 参数与证书拒绝、配对 uniform 和 legacy 行为。测试包括小规模功能运行；没有重跑正式研究语料。临时原型通过进程内导入钩子加载，未写入生产源码。^{[5][6][7][9]}

这些结果仍不是最终实施验收。当前探针不遍历全部 Python 对象、自定义迭代器、所有参数组合或历史数据版本；没有执行本次真实文件改动后的完整默认检查、独立审阅或新版本下的全部结果报告比较。正式实施必须在真实代码布局中补齐这些必要环节。

7. 风险注入和容易误判的现象

为检查探针是否能识别连带变化，另构造了一个负对照：把 schema 检查移动到基本身份检查之后。它仍然拒绝所有原来非法的情形，接受数量也仍为 979，但出现 13 项错误信息差异。对于同时含 schema_version=99 和空 config_hash 的输入，旧版先报 schema 不支持，重排后先报 config_hash 为空。只比较“是否抛异常”会漏掉这种变化。^[9]

家族责任块提取时，最初临时原型只替换 self. 属性访问，漏掉了 _validate_p4_3_record_parameters(self, parameters) 等调用中的裸 self。探针报告 939 项差异；改用名字 token 级重命名后差异归零，并额外核对展开后的 AST。这个现象支持采用明确的语法级迁移，而不是字符串替换或同时重写大量逻辑。它不是当前仓库的生产缺陷。^[9]

还观察到一个现有入口差异：直接构造的 `schema_version=2.0` 可以通过值比较并保留 `float` 类型，但 CSV 字符串 "2.0" 会被整数解析拒绝。记录这个现象的意义是固定当前接受范围，而不是在拆分时自动“修正”它。若希望统一，必须作为独立行为变更规定兼容策略和测试。^{[1][9]}

重复校验也不能随手删除。`dominated_heavy` 的维度和参数在前面的家族参数函数中检查，后面还检查证书字段。即使其中某些防御条件看起来重复，提前合并或删除都可能改变先触发的错误以及某些直接调用的前置条件。先保持整个家族分支的顺序，后续再以单独变更证明精简的收益。

旧 pickle 的成功加载同样不能视作所有记录都被重新验证。pickle 恢复和构造入口具有不同语义；不应为了得到统一流程而新增 `__setstate__`，或把旧 pickle 读取成功作为非法 CSV 拒绝的替代证据。^{[10][11]}

8. 建议实施方案

第一步固定真实实施基线和比较输入。确认前两批拆分提交及内部文档处于预期状态，将本报告中的 13 个代表、合法规范化情形和多错误组合保存为小型测试输入；保留原版本产生的 pickle。不要把全部探针输出或大规模报告文件提交到源码。

第二步实施 A：在原文件新增 `_validate_instance_family`，把原 587-805 行的完整家族责任块移入，参数为记录、已经解析的参数映射和通用证书字段集合。调用放在原家族分支的位置；参数名按语法节点或 token 迁移。schema、基础字段、JSON、比率和通用证书校验的代码顺序不动；两个规范化写入不动；不改变类定义、装饰器和公开模块身份。

第三步在真实代码中验收 A。先做旧新版差分 and 已知答案检查，再运行相关记录、家族、CSV、恢复、pickle/spawn 测试；最后执行现有默认检查和静态类型检查。独立审阅者需要实际执行有效和无效输入，并检查责任块没有丢失裸 self 引用或引入重复调用。任何结果、错误优先级或恢复行为差异都先定位，不直接更新期望结果。

第四步只在跨文件维护收益明确时实施 C。把家族参数辅助函数、家族元数据和新责任函数放入一个家族规则模块，避免类定义与规则形成运行时循环。schema 及 CSV 继续留在原文件，公共常量通过原位置重新导出。跨文件前明确私有 `get_type_hints` 的支持范围；没有决策时继续保留 A，不把隐式反射差异带入交付。

第五步再考虑每个家族的专用函数或重复条件精简。它们不属于机械迁移的验收；先观察是否真的减少常见修改需要阅读的逻辑，再决定是否值得实施。不要一次新增多个状态对象、字段模型、注册器或校验 DSL。

文档随实际布局更新：说明类与校验规则各自归属、直接构造和 CSV 的顺序、历史 DUAL 的隔离复现入口，以及本批实际检查结果。已发布报告、固定证据引用和原设计不改写。代码按本地独立提交交付，发布与合并另遵循明确授权。

9. 验收重点、缺口和停止条件

现有测试为公开字段、CSV 标题、类型路径、只读行为和序列化提供了良好基础。家族参数测试也包含 bool 替代整数、容量、桥接和证书限制。不过部分拒绝测试只要求 `TypeError` 或 `ValueError`，并未限定先报告哪条规则；现有 pickle 测试主要使用当前类型创建对象再读取，不能完全替代本批迁移前捕获的旧对象。^{[5][6][7]}

因此新增测试应集中补四类缺口：多错误并存时的首错；两个规范化写入的位置与调用者输入不变；真实旧 pickle 在新布局中的读取；新模块冷导入和类型解析边界。不为标题、文件行数或固定函数数量新增门禁，也不把全部 8,790 项探索性探针固化为永久测试。

风险	必须观察的行为	失败时处理
类型或序列化身份漂移	公开别名、字段、slots、签名、旧 pickle	停止迁移，恢复类型边界
校验顺序改变	多错误输入的首个异常类型和信息	比较责任块顺序，不改期望掩盖差异
规范化变化	JSON 格式、tuple 转换、CSV 字节、输入未变异	保留原写入时点和格式
家族资格改变	legacy、P4.3、paired uniform、custom 的接受/拒绝	分家族定位，禁止引入统一默认规则
参数与证书前置条件错位	缺字段、错误类型、维度和证书交叉约束	先恢复既有顺序，另议精简
跨模块依赖问题	冷导入、静态类型、明确的私有反射约定	保留同文件方案，补齐边界决策
恢复或研究引用变化	已有 CSV 重建、历史源码检查	使用原冻结源码，不改研究版本标签

正式完成标准是：选定责任边界更清楚，公开契约与约定的拒绝行为相同，默认检查和独立审阅通过，文档与执行入口一致。文件变短、测试总数增多或所有“正常数据”通过都不能单独作为完成依据。

如果实施要求放宽或收紧合法输入、改公开字段、改变 CSV schema、调整证书含义，或需要覆盖并未讨论的外部调用，应及时重新对齐范围。无法保证某类旧记录兼容时，交付明确限制或保留旧入口，不做无提示迁移。

10. 证据来源与复查入口

研究源码基线固定为 2c4c3de。原始输入、脚本、完整观测、旧 pickle 和复现所需源码已补充归档到受保护的 `codex/evidence/instance-contracts-study-v1` 分支，固定提交 0709c32。2026-09-15 发布时已回读远端提交与文件树；所有链接使用完整提交 ID，复查不依赖本地 `ignored` 目录。

按 [复现说明](#) 使用 Python 3.12.14 运行 `reproduce.py`，入口在新目录恢复源码和输入，自动设置 `PYTHONHASHSEED=0`，不覆盖归档。2026-09-15 已重现五个版本各 8,790 项调用、A/B/C 零差异、负对照 13 项差异，以及基线/A/C 各 15 项现有测试。旧负对照观测缺少两项后来增加的类型注解诊断；比较只允许这两个新增字段，其余历史字段全部核对。

`probe.py` 的 `baseline`、`inplace`、`stage1`、`stage2` 模式分别对应原实现和三个提取原型；`reordered` 为负对照。它只在进程中替换模块加载内容。`run_checks.py` 执行选取的现有测试；这里的原型结果不等于生产文件已修改或整套 CI 已完成。早期漏改裸 `self` 的失败观测一并归档，但该中间原型源码未保留，不能据此重放该次失败；本次也未重新执行所有历史人工 AST 核对。正式代码实施应从本节固定输入构建新旧对照，不覆盖原始观测。

[1] `greedy-failure-structures`, `_instance_contracts.py`, 本地 2c4c3de; 重点为 22-43、54-338、341-805、807-952 行。提供职责、顺序、规范化、类型与 CSV 的一手实现证据。

[2] `greedy-failure-structures`, `contracts.py` 与 包入口, 本地 2c4c3de; 提供公开导出及类型身份路径。

[3] `greedy-failure-structures`, `benchmark_planning.py`, 本地 2c4c3de; `_instance_record` 及其生成器、证书和配对来源处理。

[4] `greedy-failure-structures`, `benchmark_artifacts.py` 与 输出验证器, 本地 2c4c3de; 提供规范 CSV、恢复读取和验证调用链。

[5] `greedy-failure-structures`, `test_contracts_compatibility.py` 与 既有实例夹具, 本地 2c4c3de; 提供公开类型、列顺序、pickle/spawn 和固定记录。

[6] `greedy-failure-structures`, `test_p4_new_families.py`, 本地 2c4c3de; 重点为 983-1269 行的记录、配对、参数和证书行为。

[7] `greedy-failure-structures`, `test_p4_adversarial.py`, 本地 2c4c3de; `legacy/v2`、证书和 `construction_version` 的行为案例。

[8] `greedy-failure-structures`, `r4_dual_io.py` 与 DUAL 使用说明, 本地 2c4c3de; 提供冻结源码检查和历史复现入口。

[9] 2026-09-10 原始研究材料, 2026-09-15 补充远端归档: `summary.json`、`probe.py`、现有测试入口。同一固定提交包含输入、各版本完整观测、失败观测和负对照，以及基线源码包和新目录复现入口。

[10] Python Software Foundation, [dataclasses - Python 3.12.14 documentation](#), 访问于 2026-09-10; 字段顺序、frozen/slots、post-init 和 replace。同步核对本地 Python 3.12.14 的 `Lib/dataclasses.py`。

[11] Python Software Foundation, [pickle - Pickling Class Instances](#), 访问于 2026-09-10; 类定位和恢复对象状态的语义。

[12] Python Software Foundation, [typing - TYPE_CHECKING](#), 访问于 2026-09-10; 运行时与静态类型检查的区别。本地另核对 `get_type_hints` 的命名空间行为。